

Software Re-Engineering

CS4118

Intro to Software Re-Engineering

Lect 1,2

A Fairy Tale

- Once upon a time there was a Good Software Engineer whose Customers knew exactly what they wanted. The Good Software Engineer worked very hard to design the Perfect System that would solve all the Customers' problems now and for decades. When the Perfect System was designed, implemented and finally deployed, the Customers were very happy indeed. The Maintainer of the System had very little to do to keep the Perfect System up and running, and the Customers and the Maintainer lived happily every after.

A Fairy Tale

- Why isn't real life more like this fairy tale?
- Could it be because there are no Good Software Engineers?
- Could it be because the Users don't really know what they want?
- Or is it because the Perfect System doesn't exist?

Manny Lehman and Les Belady Laws

- The Law of Continuing Change — A program that is used in a realworld environment must change, or become progressively less useful in that environment.
- The Law of Increasing Complexity — As a program evolves, it becomes more complex, and extra resources are needed to preserve and simplify its structure.

Legacy Systems

- A legacy is something valuable that you have inherited.
- It may be old-fashioned.
- may have been developed using an outdated programming language, or an obsolete development method.
- Most likely it has changed hands several times, and shows signs of many modifications and adaptations.
- you must constantly be prepared to adapt to a changing business environment.

Legacy Systems

- The software that you use to keep your business running must therefore also be adaptable.
- Legacy system can neither be replaced nor upgraded except at a high cost.

Goal of Re-Engineering

- The goal of reengineering is to reduce the complexity of a legacy system sufficiently that it can continue to be used and adapted at an acceptable cost.

Reasons of Re-Engineering

- You might want to unbundle a monolithic system so that the individual parts can be more easily marketed separately or combined in different ways.
- You might want to improve performance.
- You might want to port the system to a new platform.
- You might want to extract the design as a first step to a new implementation.
- You might want to exploit new technology, such as emerging standards or libraries, as a step towards cutting maintenance costs.
- You might want to reduce human dependencies by documenting knowledge about the system and making it easier to maintain.

Re-Engineering Life Cycle

- “Reengineering ... is the examination and alteration of a subject system to reconstitute it in a new form and the subsequent implementation of the new form.”
- “Reverse Engineering is the process of analyzing a subject system to identify the system’s components and their interrelationships and create representations of the system in another form or at a higher level of abstraction.”

Re-Engineering Life Cycle

- “Forward Engineering is the traditional process of moving from high-level abstractions and logical, implementation-independent designs to the physical implementation of a system.”

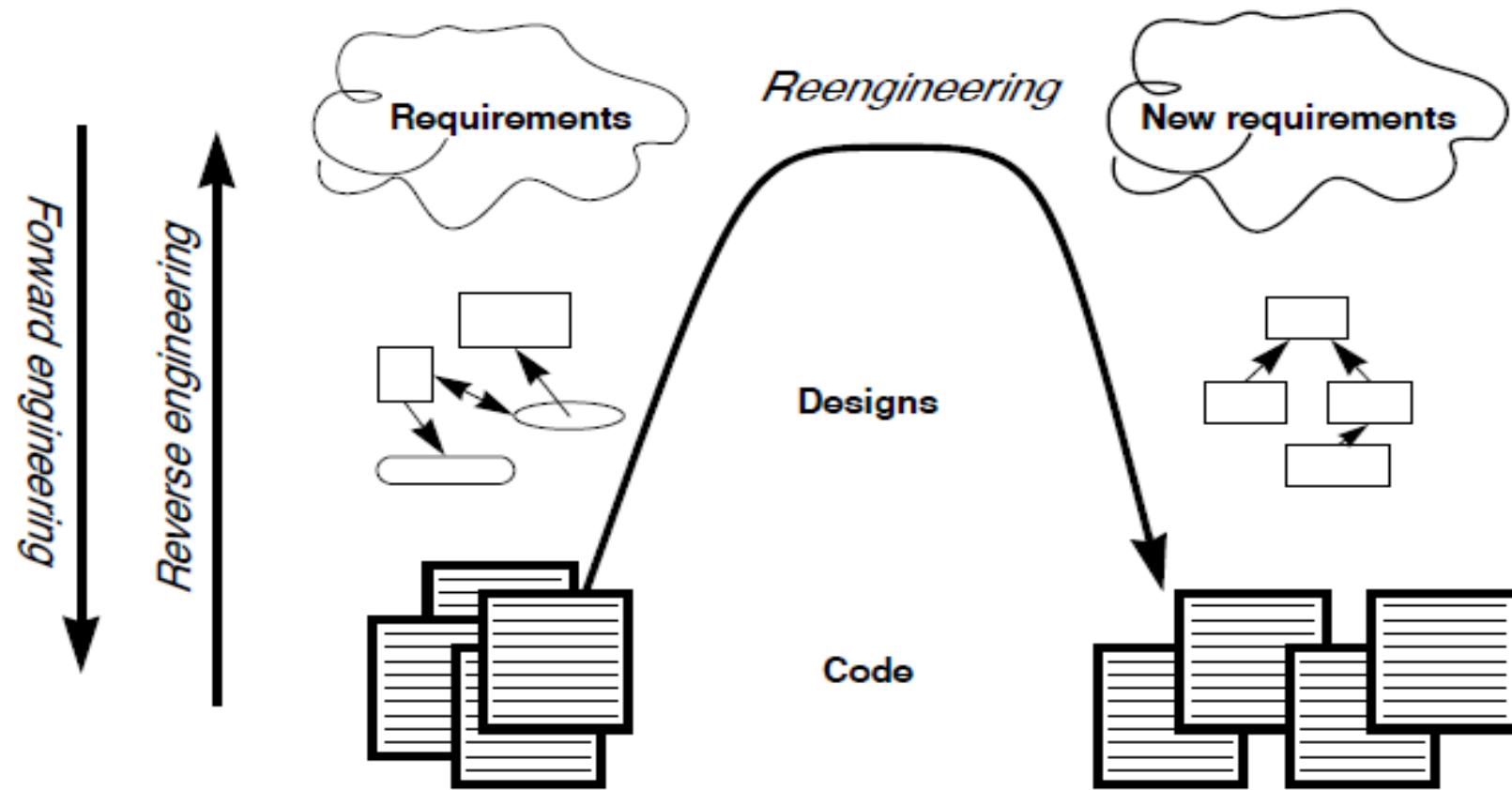
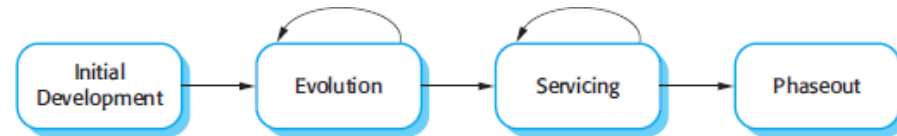


Figure 1.1: Forward, reverse and reengineering

Software Engineering challenges

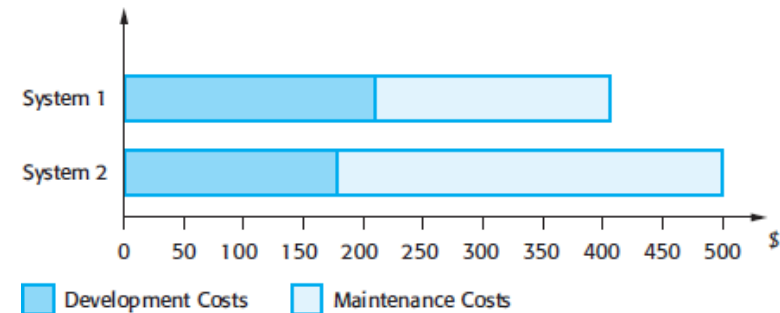
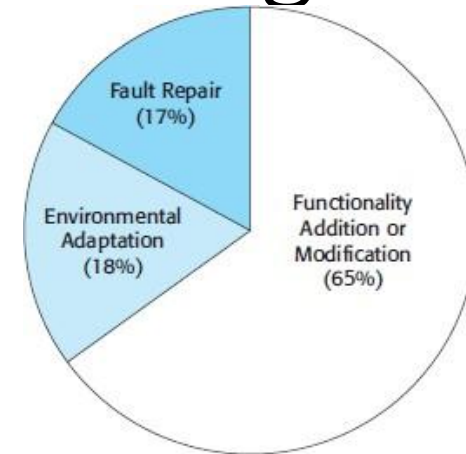


- Evolution
 - Continuous changing requirements
 - Improvement in architecture and design
 - Change management and change control procedures
 - Cost and ROI considerations
 - Lifespan

•

Software Engineering challenges

- Maintenance
 - Types
 - Fault repairs
 - Environmental adaptation
 - Functionality addition
 - Considerations and issues
 - Team stability
 - Development practices
 - Staff skills
 - Program age and structure



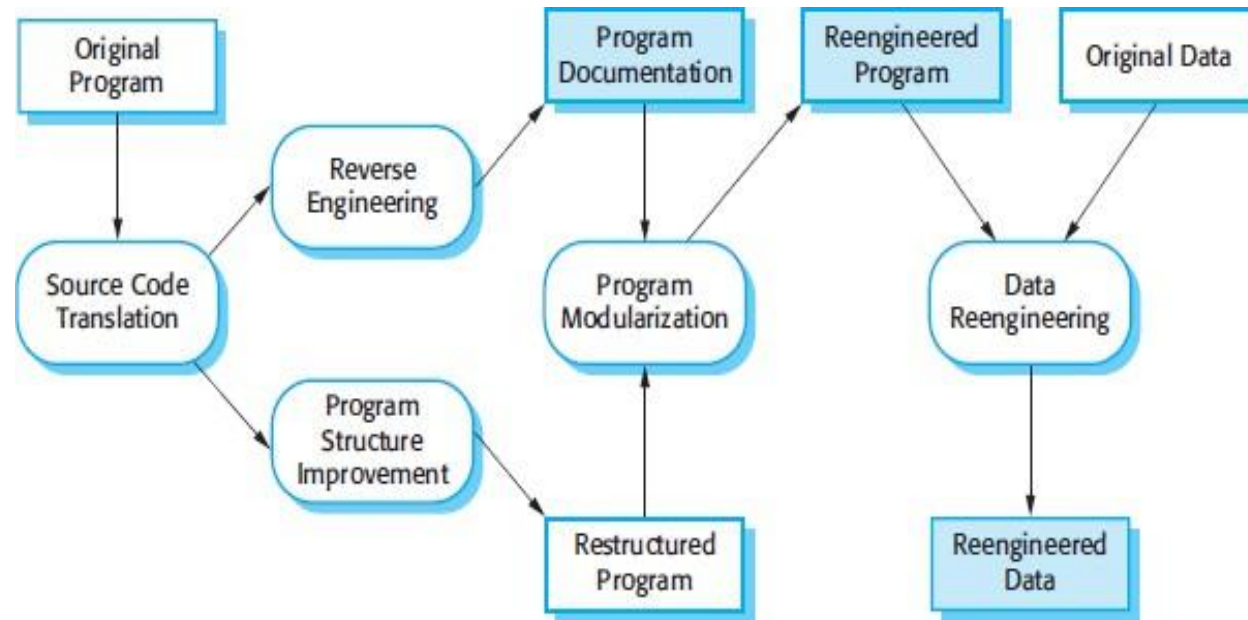
Software Engineering challenges

- Legacy systems
 - Operational and stable / reliable
 - Large-scale and complex business processes
 - Outdated software stack / obsolete technology
 - Prohibitively high costs (of replacement)
- Examples
 - Unix vs Linux
 - IE vs Edge
 - J2ME vs Android
 - Applets vs Flash vs HTML5
 - Numerous other applications in banking, retail, avionics, energy, and governance sectors

Remedial measures

- Regular maintenance, relying on data and considerations, such as:
 - Number of change requests / corrective actions
 - Average time for impact analysis and implementation
 - Number of outstanding requests
- Strategic options
 - Leave the system unchanged (with regular maintenance)
 - Scrap and replace (all or parts)
 - Re-engineer

Software Re-Engineering



- Process to improve program structure and understandability
 - Functionality remains the same
 - Improvements to architecture and design, code, documentation as well as data management are considered

Software Re-Engineering Activities

Activity	Description	Examples
Source code translation	Convert from one programming language to the other language	Java to Kotlin (Android)
	(Completely automated)	PHP to C++ (Facebook / Hiphop)
Reverse Engineering	Generating documentation (UML diagrams, API docs, etc)	Rational Rose, Star UML, Javadocs, Doxygen, etc.
	(Mostly automated with some manual intervention)	
Program structure improvement (Refactoring)	Modifications to the code (e.g. control structures, data structures, etc.)	Rename Method, Extract Method, Replace conditionals with polymorphism, etc.
	(Manual as well as IDE / Tool support)	
Program modularization	Mostly changes at the architecture and design level to remove redundancy, improve abstraction / encapsulation, etc.	Architectural patterns (MVC, MDD, etc.) and Design Patterns (State / Strategy, etc.)
	(Manual with some tool support)	
Data Re-engineering	Changes to database schema, transformations, etc.	ETL Tools (e.g. SQL Server Integration Services, etc.)
	(Mostly automated)	

Reference Material

- Read Ch 1 of OORP book